

1 Introduction

1.1 GPU Architecture Evolution

Modern NVIDIA GPUs are throughput-oriented processors designed around large numbers of parallel threads, high memory bandwidth, and hardware support for latency hiding. Their performance model differs from CPUs: instead of optimizing for a small number of low-latency cores, GPUs execute many warps concurrently and rely on massive parallelism to keep arithmetic units and memory pipelines busy. This model is effective for regular data-parallel workloads, but graph traversal stresses the architecture because memory accesses are irregular, frontiers are dynamically sized, and synchronization or atomic updates are frequently required.

The architectural progression from Ampere to Hopper and from Hopper to Blackwell is important for graph traversal because each generation improves a different part of the execution path. Tensor core improvements primarily target dense linear algebra and AI workloads, but changes to memory bandwidth, cache capacity, asynchronous data movement, atomics, synchronization, and GPU interconnects also affect irregular graph algorithms.

Ampere

Ampere GPUs, such as the NVIDIA A100, provide the baseline for many current GPU-accelerated HPC systems. Ampere introduced third-generation tensor cores, support for TF32 tensor operations, improved mixed-precision performance, Multi-Instance GPU (MIG) partitioning, and high-bandwidth HBM2e memory. For distributed workloads, A100 systems use third-generation NVLink and NVSwitch to provide high-bandwidth GPU-to-GPU communication within a node.

For graph traversal, the most relevant Ampere characteristics are high memory bandwidth, a large L2 cache, fast atomic operations relative to earlier architectures, and support for CUDA cooperative execution patterns. However, Ampere still leaves many irregular workloads limited by memory latency, random access behavior, and synchronization overhead. BFS and SSSP often perform little arithmetic per byte loaded, so they are usually constrained by memory-system efficiency rather than floating-point throughput.

Hopper Changes Relative to Ampere

Hopper GPUs, such as the NVIDIA H100, extend Ampere with features aimed at both AI and HPC. Hopper introduces fourth-generation tensor cores, FP8 tensor operations, the Transformer Engine, DPX instructions for dynamic programming, higher HBM bandwidth, larger on-chip resources, and fourth-generation NVLink. H100 NVLink provides substantially higher GPU-to-GPU bandwidth than A100 NVLink, which is important for multi-GPU and multi-node workloads where communication can dominate execution time.

One of Hopper’s important architectural additions is the thread block cluster. A thread block cluster allows multiple cooperative thread blocks to be scheduled together and to synchronize at a cluster scope. Hopper also introduces distributed shared memory within a cluster, allowing thread blocks in the same cluster to access each other’s shared memory. These features are most directly useful for algorithms that can organize work into cooperating block groups. For graph traversal, they can help with frontier processing, high-degree vertex expansion, and local aggregation before issuing global or remote updates.

Hopper also adds the Tensor Memory Accelerator (TMA), which supports efficient asynchronous movement of multidimensional data between global memory and shared memory. TMA is more naturally suited to dense tensor and tiled workloads than to pointer-chasing graph traversal, but the broader trend is important: newer GPUs increasingly provide hardware mechanisms for asynchronous data movement and overlap. Graph algorithms can benefit when pre-processing, frontier compaction, or tiled adjacency processing creates sufficiently structured memory movement.

For this thesis, Hopper is significant because it improves the hardware substrate on which NVSHMEM traversal executes. Higher memory bandwidth, stronger interconnect bandwidth, improved synchronization mechanisms, and better support for device-side execution all reduce some of the costs that appear in distributed BFS and SSSP. The algorithms remain irregular, but the penalty of remote updates and synchronization can be reduced on newer hardware.

Blackwell Changes Relative to Hopper

Blackwell GPUs, such as the NVIDIA B100, B200, and GB200-class systems, continue the trend toward larger GPU systems with stronger interconnect and memory subsystems. Blackwell introduces fifth-generation tensor cores, support for lower-precision formats such as FP4 through the second-generation Transformer Engine, higher HBM3e memory capacity and bandwidth, and fifth-generation NVLink. In GB200 systems, NVLink and NVLink Switch are designed to connect many GPUs with very high aggregate bandwidth, making the system behave more like a large coherent accelerator fabric for communication-heavy workloads.

Although many Blackwell headline features target AI training and inference, the architectural improvements also matter for graph analytics. Larger and faster memory systems help bandwidth-bound traversal kernels. Higher GPU-to-GPU bandwidth lowers the cost of remote frontier updates, parent writes, distance relaxations, and distributed work queues. Improved reliability, availability, and serviceability features are also relevant at the scale where Graph500-class problems are intended to run.

Compared with Hopper, Blackwell’s most relevant contribution for this thesis is the stronger communication and memory platform. Distributed graph traversal performance depends on the rate at which GPUs can exchange fine-grained updates and sustain irregular memory access. NVSHMEM exposes GPU-initiated remote memory operations, and Blackwell-class interconnect im-

provements increase the potential value of that model because more communication can remain on the accelerator path instead of being bottlenecked by host-mediated transfers.

Implications for Graph Traversal

The evolution from Ampere to Hopper to Blackwell shows that GPU graph traversal should not be evaluated only through peak floating-point performance. BFS and SSSP depend more heavily on memory bandwidth, cache behavior, atomic throughput, synchronization cost, and inter-GPU communication. Ampere provides strong baseline bandwidth and atomics. Hopper improves synchronization, memory movement mechanisms, and NVLink bandwidth. Blackwell further increases memory capacity, memory bandwidth, and accelerator interconnect bandwidth.

For NVSHMEM-based traversal, these changes are especially relevant. The programming model allows GPU kernels to issue remote memory operations directly, so improvements in GPU interconnects and device-side execution paths can translate into better traversal scalability. The central challenge remains irregularity: graph algorithms expose unpredictable memory references and load imbalance. Newer architectures reduce the cost of these effects but do not eliminate the need for careful frontier management, work distribution, and synchronization design.

1.2 Message Passing Interface

The Message Passing Interface (MPI) is the dominant programming model for distributed-memory high-performance computing. MPI defines a standardized API for communication between independent processes, commonly called ranks, that execute either on the same node or across multiple nodes in a cluster. Each rank owns a private address space, and data movement between ranks is expressed explicitly through message-passing operations.

MPI is important for large-scale graph traversal because the graph usually cannot be stored or processed efficiently by a single process at benchmark scale. Instead, vertices, edges, frontiers, validation data, and performance counters are distributed across ranks. The communication pattern is irregular: a traversal step may discover vertices owned by many different ranks, and the amount of communication depends on the graph structure and the current frontier rather than on a fixed stencil or dense matrix layout.

Point-to-Point Communication

MPI point-to-point operations transfer data between specific source and destination ranks. Blocking operations such as `MPI_Send` and `MPI_Recv` complete only when their communication requirements are satisfied, while nonblocking operations such as `MPI_Isend` and `MPI_Irecv` allow communication to be initiated and completed later with calls such as `MPI_Wait` or `MPI_Test`. Nonblocking

communication is useful in graph traversal because message exchange can potentially overlap with local processing of the current frontier.

Collective Communication

Collective operations involve all ranks in a communicator. Common examples include `MPI_Barrier` for synchronization, `MPI_Bcast` for broadcasting data from one rank, `MPI_Reduce` and `MPI_Allreduce` for combining values, and `MPI_Alltoall` or `MPI_Alltoallv` for exchanging variable amounts of data between ranks. Graph workloads often use collectives to compute global termination conditions, aggregate performance metrics, distribute benchmark parameters, and coordinate validation.

MPI in This Thesis

In this work, MPI provides the process model and host-side coordination around the NVSHMEM traversal kernels. The benchmark runner uses MPI to launch multiple ranks, coordinate graph generation, perform optional MPI file I/O, convert generated edge tuples into distributed graph structures, and validate BFS and SSSP results. NVSHMEM is used for GPU-side communication during traversal, while MPI remains responsible for conventional distributed-memory orchestration on the host.

This separation is useful because MPI and NVSHMEM address different layers of the system. MPI gives a portable and mature interface for rank management and host communication. NVSHMEM exposes a partitioned global address space for GPU kernels, enabling device-side remote memory operations without returning to the CPU for every inter-rank interaction. Together, they support a hybrid execution model in which MPI manages the benchmark environment and NVSHMEM accelerates fine-grained traversal communication on GPUs.

1.3 NVSHMEM

NVSHMEM is a GPU-oriented implementation of the OpenSHMEM partitioned global address space model. It provides one-sided communication operations that allow a processing element to read, write, or atomically update memory associated with another processing element. In an NVSHMEM program, each processing element has a symmetric memory region, and the same allocation call creates corresponding objects on all participating processing elements. This symmetry allows a remote address to be computed from a local pointer and a destination processing element identifier.

The main distinction between NVSHMEM and traditional host-driven communication is that NVSHMEM operations can be issued directly from CUDA kernels. GPU threads can perform remote memory operations without first returning control to the CPU. This feature is important for irregular algorithms because the communication target is often discovered dynamically by the GPU

while processing data. In graph traversal, a thread may inspect an edge, determine that the neighbor vertex is owned by another rank, and immediately update remote state through NVSHMEM.

Partitioned Global Address Space

The partitioned global address space model combines local memory ownership with a global addressing abstraction. Each processing element owns its local portion of memory, but remote memory can be accessed through one-sided operations. Unlike two-sided message passing, the remote side does not need to post a matching receive operation for each individual transfer. This makes the model attractive for fine-grained and data-dependent communication patterns.

In the context of distributed graph traversal, vertex metadata such as parent arrays, distance arrays, frontier state, or work queues can be partitioned across processing elements. When traversal discovers a vertex owned by a remote processing element, NVSHMEM operations can update that remote vertex state directly. Atomic operations are especially important because several GPU threads or ranks may discover the same vertex concurrently.

Device-Side Communication

NVSHMEM supports communication from both host code and CUDA device code. Device-side communication avoids the overhead of repeatedly synchronizing with the CPU during traversal. This is useful for breadth-first search and single-source shortest paths because both algorithms repeatedly expand frontiers whose size and destination distribution are not known in advance.

For BFS, device-side remote atomics can be used to claim a parent slot when a vertex is first discovered. For SSSP, remote atomics or compare-and-update patterns can be used to relax distance values. These operations map naturally to graph traversal because the algorithm’s communication granularity is often a vertex or edge update rather than a large contiguous bulk message.

NVSHMEM in This Thesis

This thesis uses NVSHMEM to implement distributed GPU graph traversal kernels. MPI remains responsible for launching ranks and coordinating host-side benchmark behavior, while NVSHMEM enables GPU kernels to communicate across ranks during BFS and SSSP. This design reduces dependence on CPU-mediated communication in the traversal loop and aligns the communication mechanism with the hardware executing the graph algorithm.

The expected benefit is improved support for irregular, fine-grained communication on GPU clusters. Graph500-style graphs have skewed degree distributions, poor locality, and unpredictable frontier expansion. NVSHMEM gives the traversal kernels a mechanism to express these remote updates directly from the GPU, which is a better match for the dynamic behavior of graph algorithms than repeatedly staging all communication through host-side MPI calls.

1.4 Parallel Breadth-First Search

Breadth-first search (BFS) is a fundamental graph traversal algorithm that discovers vertices in nondecreasing order of their distance, measured in number of edges, from a chosen source vertex. In the sequential algorithm, BFS maintains a frontier of vertices at the current level. The algorithm inspects the outgoing edges of the current frontier, discovers previously unvisited neighbors, assigns their parent and level, and places them in the next frontier. This process repeats until the frontier becomes empty.

Parallel BFS exposes concurrency because all vertices in the same frontier level can be processed independently. Each thread or processing element can inspect a subset of frontier vertices and their incident edges. The main challenge is coordination when multiple workers discover the same neighbor concurrently. A correct implementation must ensure that each vertex is inserted into the BFS tree only once and that the assigned parent connects it to a vertex from the previous level. In practice, this is commonly handled with atomic compare-and-swap operations on the parent array or visited state.

Frontier Expansion

The frontier representation strongly affects BFS performance. A top-down BFS expands the current frontier by visiting the outgoing edges of active vertices. This approach is efficient when the frontier is small because work is proportional to the number of edges incident to active vertices. A bottom-up BFS instead scans undiscovered vertices and checks whether any neighbor belongs to the current frontier. This approach can reduce work when the frontier is large, especially in low-diameter graphs where a large fraction of the graph becomes active in a few levels. Hybrid BFS implementations switch between these modes depending on frontier size and graph structure.

On GPUs, frontier expansion must expose enough parallelism to fill the device while avoiding excessive duplicate work and memory contention. High-degree vertices can create load imbalance because one frontier vertex may have orders of magnitude more edges than another. Parallel implementations often distribute work at the edge level, warp level, or block level rather than assigning one thread per vertex. This improves load balance but increases the need for efficient atomic updates and compact frontier construction.

Distributed BFS

In distributed-memory BFS, the graph is partitioned across ranks or processing elements. Each process owns a subset of vertices and the associated local graph data. When a traversal step discovers a neighbor owned by another process, the discovery must be communicated to the owner so that the remote parent or visited state can be updated. This communication is irregular because the set of destination processes depends on the current frontier and the edge distribution.

For an MPI-only implementation, discovered remote vertices are typically buffered and exchanged between ranks using point-to-point or collective com-

munication. With NVSHMEM, GPU threads can directly issue remote memory operations while expanding the frontier. This allows the traversal kernel to update remote parent state from the device, reducing the need to return to the CPU after every level or communication phase. The correctness condition remains the same: each reachable vertex must receive a valid parent at the previous BFS level, and vertices not reached from the source remain unvisited.

1.5 Single-Source Shortest Paths

The single-source shortest paths (SSSP) problem computes the minimum path distance from one source vertex to every other reachable vertex in a weighted graph. For graphs with non-negative edge weights, the classical sequential algorithm is Dijkstra’s algorithm. It repeatedly selects the unsettled vertex with the smallest tentative distance, relaxes its outgoing edges, and updates neighboring tentative distances when a shorter path is found.

SSSP is more difficult to parallelize than BFS because the ordering of work depends on path weights rather than only on traversal depth. BFS advances level by level using unit edge costs, while SSSP must respect numeric distance values. A vertex may be discovered early with a tentative distance and later improved by a shorter path. Therefore, parallel SSSP implementations commonly use relaxation-based methods that allow repeated updates until distances converge.

Edge Relaxation

The core operation in SSSP is edge relaxation. For an edge from vertex u to vertex v with weight w , the algorithm checks whether $\text{distance}[u] + w$ is smaller than the current $\text{distance}[v]$. If it is smaller, $\text{distance}[v]$ is updated and u becomes the current predecessor or parent of v . In a parallel implementation, many edges can be relaxed concurrently, but concurrent relaxations may target the same destination vertex. Atomic minimum operations or compare-and-update loops are used to preserve the smallest discovered distance.

The parent array in SSSP records a shortest-path tree. Unlike BFS, parent assignment is tied to the best distance value rather than the first discovery. If a shorter path is found later, both the distance and parent should be updated consistently. Correctness requires that every reachable vertex has a distance equal to the shortest path from the source and that each parent edge is part of a valid shortest path.

Parallel Work Management

Parallel SSSP algorithms often maintain an active set of vertices whose outgoing edges may improve neighboring distances. When a vertex distance decreases, the vertex is inserted into a future worklist so that its outgoing edges can be relaxed again. This approach resembles frontier-based BFS, but the same vertex may become active multiple times as shorter paths are discovered. The amount

of work is therefore sensitive to graph weights, update ordering, and contention on high-degree or frequently relaxed vertices.

On GPUs, SSSP benefits from exposing large numbers of edge relaxations in parallel, but it also suffers from irregular memory access and atomic contention. In distributed memory, relaxations may target vertices owned by remote ranks. NVSHMEM enables device-side remote updates to distributed distance and parent arrays, allowing the GPU traversal kernel to communicate improvements directly. Termination requires a global condition: the algorithm is complete only when no rank has active work and no in-flight remote update can produce additional work.

1.6 Graph 500

Benchmark Specification

Graph 500 Benchmarks 1 (“Search”) and 2 (“Shortest Path”)

Brief Description of the Graph 500 Benchmark

Data-intensive supercomputer applications are an increasingly important workload, but are ill-suited for platforms designed for 3D physics simulations. Application performance cannot be improved without a meaningful benchmark. Graphs are a core part of most analytics workloads. Backed by a steering committee of over 30 international HPC experts from academia, industry, and national laboratories, this specification establishes a large-scale benchmark for these applications. It will offer a forum for the community and provide a rallying point for data-intensive supercomputing problems. This is the first serious approach to augment the Top 500 with data-intensive applications.

The intent of benchmark problems (“Search” and “Shortest-Path”) is to develop a compact application that has multiple analysis techniques (multiple kernels) accessing a single data structure representing a weighted, undirected graph. In addition to a kernel to construct the graph from the input tuple list, there are two additional computational kernels to operate on the graph.

This benchmark includes a scalable data generator which produces edge tuples containing the start vertex and end vertex for each edge. The first kernel constructs an undirected graph in a format usable by all subsequent kernels. No subsequent modifications are permitted to benefit specific kernels. The second kernel performs a breadth-first search of the graph. The third kernel performs multiple single-source shortest path computations on the graph. All three kernels are timed.

There are five problem classes defined by their input size:

toy 17GB or around 1010 bytes, which we also call level 10, mini 140GB (1011 bytes, level 11), small 1TB (1012 bytes, level 12), medium 17TB (1013 bytes, level 13), large 140TB (1014 bytes, level 14), and huge 1.1PB (1015 bytes, level 15).

Table classes provides the parameters used by the graph generator specified below.

References D.A. Bader, J. Feo, J. Gilbert, J. Kepner, D. Koester, E. Loh, K. Madduri, W. Mann, Theresa Meuse, HPCS Scalable Synthetic Compact Applications #2 Graph Analysis (SSCA#2 v2.2 Specification), 5 September 2007.

Overall Benchmark

The benchmark performs the following steps:

- Generate the edge list.
 - Construct a graph from the edge list (timed, kernel 1).
 - Randomly sample 64 unique search keys with degree at least one, not counting self-loops.
 - For each search key:
 - Compute the parent array (timed, kernel 2).
 - Validate that the parent array is a correct BFS search tree for the given search tree.
 - For each search key:
 - Compute the parent array and the distance array (timed, kernel 3).
 - Validate that the parent array/distance vector is a correct SSSP search tree with shortest paths for the given search tree.
 - Compute and output performance information.
- Only the sections marked as timed are included in the performance information. Note that all uses of “random” permit pseudorandom number generation. Note that the Kernel 2 and Kernel 3 are run in separate loops and not consecutively off the same initial vertex. Kernel 2 and Kernel 3 can be run on graphs of different scales that are generated by separate runs of Kernel 1.

Generating the Edge List

Brief Description The scalable data generator will construct a list of edge tuples containing vertex identifiers. Each edge is undirected with its endpoints given in the tuple as StartVertex, EndVertex and Weight. If the edge tuples are only to be used for running Kernel 2, it is permissible to not generate edge weights. This allows BFS runs that are not encumbered by unnecessary memory usage resulting from storing edge weights.

The intent of the first kernel below is to convert a list with no locality into a more optimized form. The generated list of input tuples must not exhibit any locality that can be exploited by the computational kernels. Thus, the vertex numbers must be randomized and a random ordering of tuples must be presented to kernel 1. The data generator may be parallelized, but the vertex names must be globally consistent and care must be taken to minimize effects of data locality at the processor level.

Detailed Text Description The edge tuples will have the form <StartVertex, EndVertex, Weight> where StartVertex is one endpoint vertex label and EndVertex is the other endpoint vertex label, and Weight is the weight of the

edge. The space of labels is the set of integers beginning with zero up to but not including the number of vertices N (defined below), and the space of weights is the range $[0,1)$ of single precision floats. The kernels are not provided the size N explicitly but must discover it if required for constructing the graph.

The benchmark takes only one parameter as input:

SCALE The logarithm base two of the number of vertices. $\text{edgefactor} = 16$ The ratio of the graph’s edge count to its vertex count (i.e., half the average degree of a vertex in the graph).

These inputs determine the graph’s size:

N the total number of vertices, **2SCALE**. An implementation may use any set of N distinct integers to number the vertices, but at least 48 bits must be allocated per vertex number and 32 bits must be allocated for edge weight unless benchmark is run in BFS-only mode. Other parameters may be assumed to fit within the natural word of the machine. N is derived from the problem’s scaling parameter. M the number of edges. $M = \text{edgefactor} * N$.

The graph generator is a Kronecker generator similar to the Recursive Matrix (R-MAT) scale-free graph generation algorithm [Chakrabarti, et al., 2004]. For ease of discussion, the description of this R-MAT generator uses an adjacency matrix data structure; however, implementations may use any alternate approach that outputs the equivalent list of edge tuples. This model recursively sub-divides the adjacency matrix of the graph into four equal-sized partitions and distributes edges within these partitions with unequal probabilities. Initially, the adjacency matrix is empty, and edges are added one at a time. Each edge chooses one of the four partitions with probabilities A , B , C , and D , respectively. These probabilities, the initiator parameters, are provided in Table initiator. The weight is chosen randomly with uniform distribution from the interval of $[0, 1)$.

Initiator parameters for the Kronecker graph generator

$A = 0.57$ $B = 0.19$

$C = 0.19$ $D = 1-(A+B+C) = 0.05$

The next section details a high-level implementation for this generator. High-performance, parallel implementations are included in the reference implementation.

The graph generator creates a small number of multiple edges between two vertices as well as self-loops. Multiple edges, self-loops, and isolated vertices may be ignored in the subsequent kernels but must be included in the edge list provided to the first kernel. The algorithm also generates the data tuples with high degrees of locality. Thus, as a final step, vertex numbers must be randomly permuted, and then the edge tuples randomly shuffled.

It is permissible to run the data generator in parallel. In this case, it is necessary to ensure that the vertices are named globally, and that the generated data does not possess any locality, either in local memory or globally across processors.

The scalable data generator should be run before starting kernel 1, storing its results to either RAM or disk. If stored to disk, the data may be retrieved

before starting kernel 1. The data generator and retrieval operations need not be timed.

Sample High-Level Implementation of the Kronecker Generator The GNU Octave routine in Algorithm generator is an attractive implementation in that it is embarrassingly parallel and does not require the explicit formation of the adjacency matrix.

```
function ijw = kronecker_generator (SCALE, edgefactor)
%% Generate an edgelist according to the Graph500 parameters. In this
%% sample, the edge list is returned in an array with three rows,
%% where StartVertex is first row, EndVertex is the second row, and
%% Weight is the third row. The vertex labels start at zero.
%%
%% Example, creating a sparse matrix for viewing:
%% ijw = kronecker_generator (10, 16);
%% G = sparse (ijw(1,:)+1, ijw(2,:)+1, ones (1, size (ijw, 2)));
%% spy (G);
%% The spy plot should appear fairly dense. Any locality
%% is removed by the final permutations.

%% Set number of vertices.
N = 2^SCALE;

%% Set number of edges.
M = edgefactor * N;

%% Set initiator probabilities.
[A, B, C] = deal (0.57, 0.19, 0.19);

%% Create index arrays.
ijw = ones (3, M);
%% Loop over each order of bit.
ab = A + B;
c_norm = C/(1 - (A + B));
a_norm = A/(A + B);

for ib = 1:SCALE,
%% Compare with probabilities and set bits of indices.
ii_bit = rand (1, M) > ab;
jj_bit = rand (1, M) > ( c_norm * ii_bit + a_norm * not (ii_bit) );
ijw(1:2,:) = ijw(1:2,:) + 2^(ib-1) * [ii_bit; jj_bit];
end

%% Generate weights
ijw(3,:) = unifrnd(0, 1, M);

%% Permute vertex labels
p = randperm (N);
ijw(1:2,:) = p(ijw(1:2,:));

%% Permute the edge list
p = randperm (M);
ijw = ijw(:, p);

%% Adjust to zero-based labels.
```

```

ijw(1:2,:) = ijw(1:2,:) - 1;

endfunction

```

Parameter Settings The input parameter settings for each class are given in Table classes.

Problem class	Scale	Edge factor	Size in TB
(BFS, 64 bits/edge)	Size in TB		
(BFS, 48 bits/edge)	Size in TB		
(SSSP, 48bits+32bits/edge)			
Toy (level 10)	26	16	0.017 0.013 0.022
Mini (level 11)	29	16	0.137 0.103 0.172
Small (level 12)	32	16	1.100 0.825 1.374
Medium (level 13)	36	16	17.592 13.194 21.990
Large (level 14)	39	16	140.738 105.553 175.921
Huge (level 15)	42	16	1125.900 844.425 1407.375

References D. Chakrabarti, Y. Zhan, and C. Faloutsos, R-MAT: A recursive model for graph mining, SIAM Data Mining 2004.

Section 17.6, Algorithms in C (third edition). Part 5 Graph Algorithms, Robert Sedgwick (Programs 17.7 and 17.8)

P. Sanders, Random Permutations on Distributed, External and Hierarchical Memory, Information Processing Letters 67 (1988) pp 305-309.

Kernel 1 – Graph Construction

Description The first kernel may transform the edge list to any data structures (held in internal or external memory) that are used for the remaining kernels. For instance, kernel 1 may construct a (sparse) graph from a list of tuples; each tuple contains endpoint vertex identifiers for an edge, and a weight that represents data assigned to the edge.

The graph may be represented in any manner, but it may not be modified by or between subsequent kernels. Space may be reserved in the data structure for marking or locking. Only one copy of a kernel will be run at a time; that kernel has exclusive access to any such marking or locking space and is permitted to modify that space (only).

There are various internal memory representations for sparse graphs, including (but not limited to) sparse matrices and (multi-level) linked lists. For the purposes of this application, the kernel is provided only the edge list and the edge list's size. Further information such as the number of vertices must be computed within this kernel. Algorithm kernel1 provides a high-level sample implementation of kernel 1.

The process of constructing the graph data structure (in internal or external memory) from the set of tuples must be timed.

```

function G = kernel_1 (ij)
%% Compute a sparse adjacency matrix representation
%% of the graph with edges from ij.

%% Remove self-edges.
ij(:, ij(1,:) == ij(2,:)) = [];
%% Adjust away from zero labels.
ij = ij + 1;
%% Find the maximum label for sizing.
N = max (max (ij));
%% Create the matrix, ensuring it is square.
G = sparse (ij(1,:), ij(2,:), ones (1, size (ij, 2)), N, N);
%% Symmetrize to model an undirected graph.
G = spones (G + G.');
```

References Section 17.6 Algorithms in C third edition Part 5 Graph Algorithms, Robert Sedgewick (Program 17.9)

Sampling 64 Search Keys

The search keys must be randomly sampled from the vertices in the graph. To avoid trivial searches, sample only from vertices that are connected to some other vertex. Their degrees, not counting self-loops, must be at least one. If there are fewer than 64 such vertices, run fewer than 64 searches. This should never occur with the graph sizes in this benchmark, but there is a non-zero probability of producing a trivial or nearly trivial graph. The number of search keys used is included in the output, but this step is untimed.

Kernel 2 – Breadth-First Search

Description A Breadth-First Search (BFS) of a graph starts with a single source vertex, then, in phases, finds and labels its neighbors, then the neighbors of its neighbors, etc. This is a fundamental method on which many graph algorithms are based. A formal description of BFS can be found in Cormen, Leiserson, and Rivest. Below, we specify the input and output for a BFS benchmark, and we impose some constraints on the computation. However, we do not constrain the choice of BFS algorithm itself, as long as it produces a correct BFS tree as output.

This benchmark’s memory access pattern (internal or external) is data-dependent with small average prefetch depth. As in a simple concurrent linked-list traversal benchmark, performance reflects an architecture’s throughput when executing concurrent threads, each of low memory concurrency and high memory reference density. Unlike such a benchmark, this one also measures resilience to hot-spotting when many of the memory references are to the same location; efficiency when every thread’s execution path depends on the asynchronous side-effects of others; and the ability to dynamically load balance unpredictably sized work units. Measuring synchronization performance is not a primary goal here.

You may not search from multiple search keys concurrently. No information can be passed between different invocations of this kernel. The kernel may return a depth array to be used in validation.

ALGORITHM NOTE We allow a benign race condition when vertices at BFS level k are discovering vertices at level $k+1$. Specifically, we do not require synchronization to ensure that the first visitor must become the parent while locking out subsequent visitors. As long as the discovered BFS tree is correct at the end, the algorithm is considered to be correct.

Kernel 2 Output For each search key, the routine must return an array containing valid breadth-first search parent information (per vertex). The parent of the search key is itself, and the parent of any vertex not included in the tree is -1. Algorithm `kernel2` provides a sample (and inefficient) high-level implementation of kernel two.

```
function parent = kernel_2 (G, root)
%% Compute a sparse adjacency matrix representation
%% of the graph with edges from ij.

N = size (G, 1);
%% Adjust from zero labels.
root = root + 1;
parent = zeros (N, 1);
parent (root) = root;

vlist = zeros (N, 1);
vlist(1) = root;
lastk = 1;
for k = 1:N,
v = vlist(k);
if v == 0, break; end
[I,J,V] = find (G(:, v));
nxt = I(parent(I) == 0);
parent(nxt) = v;
vlist(lastk + (1:length (nxt))) = nxt;
lastk = lastk + length (nxt);
end

%% Adjust to zero labels.
parent = parent - 1;
```

Kernel 3 – Single Source Shortest Paths

Description A single-source shortest paths (SSSP) computation finds the shortest distance from a given starting vertex to every other vertex in the graph. A formal description of SSSP on graphs with non-negative weights also can be found in Cormen, Leiserson, and Rivest. We specify the input and output for a SSSP benchmark, and we impose some constraints on the computation. However, we do not constrain the choice of SSSP algorithm itself, as long as the implementation produces a correct SSSP distance vector and parent tree as

output. This is a separate kernel and cannot use data computed by Kernel 2 (BFS).

This kernel extends the overall benchmark with additional tests and data access per vertex. Many but not all algorithms for SSSP are similar to BFS and suffer from similar issues of hot-spotting and duplicate memory references.

You may not search from multiple initial vertices concurrently. No information can be passed between different invocations of this kernel.

ALGORITHM NOTE We allow benign race conditions within SSSP as well. We do not require that a first visitor must prevent subsequent visitors from taking the parent slot. As long as the SSSP distances and parent tree are correct at the end, the algorithm is considered to be correct.

Kernel 3 Output For each initial vertex, the routine must return a the distance of each vertex from the initial vertex and the parent of each vertex in a valid single-source shortest path tree. The parent of the initial vertex is itself, and the parent of any vertex not included in the tree is -1. Algorithm below provides a sample high-level implementation of Kernel 3.

```
function [parent, d] = kernel_3 (G, root)
%% Compute the shortest path lengths and parent
%% tree starting from vertex root on the graph
%% represented by the sparse matrix G. Every
%% vertex in G can be reached from root.

N = size (G, 1);
%% Adjust from zero labels.
root = root + 1;
d = inf * ones (N, 1);
parent = zeros (N, 1);
d (root) = 0;
parent (root) = root;

Q = 1:N;
while length (Q) > 0
    [dist, idx] = min (d(Q));
    v = Q(idx);
    Q = setdiff (Q, v);
    [I, J, V] = find (G (:, v));
    for idx = 1:length(I),
        u = I(idx);
        dist_tmp = d(v) + V(idx);
        if dist_tmp < d(u),
            d(u) = dist_tmp;
            parent(u) = v;
        end
    end
end

%% Adjust back to zero labels.
parent = parent - 1;
```

References The Shortest Path Problem: Ninth DIMACS Implementation Challenge. C. Demetrescu, A.V. Goldberg, and D.S. Johnson, eds. DIMACS series in discrete mathematics and theoretical computer science, American Mathematical Society, 2009.

9th DIMACS Implementation Challenge – Shortest Paths.

Validation

It is not intended that the results of full-scale runs of this benchmark can be validated by exact comparison to a standard reference result. At full scale, the data set is enormous, and its exact details depend on the pseudo-random number generator and BFS or SSSP algorithm used. Therefore, the validation of an implementation of the benchmark uses soft checking of the results.

We emphasize that the intent of this benchmark is to exercise these algorithms on the largest data sets that will fit on machines being evaluated. However, for debugging purposes it may be desirable to run on small data sets, and it may be desirable to verify parallel results against serial results, or even against results from the executable specification.

The executable specification verifies its results by comparing them with results computed directly from the tuple list.

The validation procedure for BFS (Kernel 2) is similar to one from version 1.2 of the benchmark. The validation procedure for SSSP (Kernel 3) constructs search depth tree in place of distance array and then runs SSSP validation routine. After each search, run (but do not time) a function that ensures that the discovered parent tree and distance vector are correct by ensuring that:

- the BFS/SSSP tree is a tree and does not contain cycles,
 - each tree edge connects vertices whose
 - a) BFS levels differ by exactly one,
 - b) SSSP distances differ by at most the weight of the edge,
 - every edge in the input list has vertices with
 - a) BFS levels that differ by at most one or that both are not in the BFS tree,
 - b) SSSP distances that differ by at most the weight of the edge or are not in the SSSP tree,
 - the BFS/SSSP tree spans an entire connected component's vertices, and a node and its BFS/SSSP parent are joined by an edge of the original graph.
- Algorithm validate shows a sample validation routine.

```
function out = validate (parent, ijw, search_key, d, is_sssp)
%% Validate the results of BFS or SSSP.

%% Default: no error.
out = 1;

%% Adjust from zero labels.
parent = parent + 1;
search_key = search_key + 1;
ijw = ijw + 1;
```



```

%% Remove self-loops.
ijw(:,(ijw(1, :) == ijw(2, :)))' = [];

%% root must be the parent of itself.
if parent (search_key) != search_key,
out = 0;
return;
end

N = max (max (ijw(1:2,:)));
slice = find (parent > 0);

%% Compute levels and check for cycles.
level = zeros (size (parent));
level (slice) = 1;
P = parent (slice);
mask = P != search_key;
k = 0;
while any (mask),
level(slice(mask)) = level(slice(mask)) + 1;
P = parent (P);
mask = P != search_key;
k = k + 1;
if k > N,
%% There must be a cycle in the tree.
out = -3;
return;
end
end

%% Check that there are no edges with only one end in the tree.
%% This also checks the component condition.
lij = level (ijw(1:2,:));
neither_in = lij(1,:) == 0 & lij(2,:) == 0;
both_in = lij(1,:) > 0 & lij(2,:) > 0;
if any (not (neither_in | both_in)),
out = -4;
return
end

%% Validate the distances/levels.
respects_tree_level = true(1,size(ijw, 2));
if !is_sssp
respects_tree_level = abs (lij(1,:) - lij(2,:)) <= 1;
else
respects_tree_level = abs (d(ijw(1,:)) - d(ijw(2,:)))' <= ijw(3,:);
end
if any (not (neither_in | respects_tree_level))
out = -5;
return
end

```

Computing and Outputting Performance Information

Timing Start the time for a search immediately prior to visiting the search root. Stop the time for that search when the output has been written to memory.

Do not time any I/O outside of the search routine. The spirit of the benchmark is to gauge the performance of a single search. We run many searches in order to compute means and variances, not to amortize data structure setup time.

Performance Metric (TEPS) In order to compare the performance of Graph 500 “Search” implementations across a variety of architectures, programming models, and productivity languages and frameworks, we adopt a new performance metric described in this section. In the spirit of well-known computing rates floating-point operations per second (flops) measured by the LINPACK benchmark and global updates per second (GUPs) measured by the HPCC RandomAccess benchmark, we define a new rate called traversed edges per second (TEPS). We measure TEPS through the benchmarking of Kernel 2 and Kernel 3 as follows. Let $\text{timeK2}(n)$ be the measured execution time for a kernel run. Let m be the number of undirected edges in a traversed component of the graph counted as number of self-loop edge tuples within component traversed added to halved number of non self-loop edge tuples within component traversed. We define the normalized performance rate (number of edge traversals per second) as:

$$\text{TEPS}(n) = m / \text{timeK2}(n)$$

Output The output must contain the following information:

SCALE Graph generation parameter edgefactor Graph generation parameter
 NBFS Number of BFS searches run, 64 for non-trivial graphs construction_time
 The single kernel 1 time bfs_min_time, bfs_firstquartile_time, bfs_median_time,
 bfs_thirdquartile_time, bfs_max_time Quartiles for the kernel 2 times bfs_mean_time,
 bfs_stddev_time Mean and standard deviation of the kernel 2 times bfs_min_nedge,
 bfs_firstquartile_nedge, bfs_median_nedge, bfs_thirdquartile_nedge, bfs_max_nedge
 Quartiles for the number of input edges visited by kernel 2, see TEPS section above.
 bfs_mean_nedge, bfs_stddev_nedge Mean and standard deviation of the number of input edges visited by kernel 2, see TEPS section above.
 bfs_min_TEPS, bfs_firstquartile_TEPS, bfs_median_TEPS, bfs_thirdquartile_TEPS,
 bfs_max_TEPS Quartiles for the kernel 2 TEPS bfs_harmonic_mean_TEPS,
 bfs_harmonic_stddev_TEPS Mean and standard deviation of the kernel 2
 TEPS. sssp_min_time, sssp_firstquartile_time, sssp_median_time, sssp_thirdquartile_time,
 sssp_max_time Quartiles for the kernel 3 times sssp_mean_time, sssp_stddev_time
 Mean and standard deviation of the kernel 3 times sssp_min_nedge, sssp_firstquartile_nedge,
 sssp_median_nedge, sssp_thirdquartile_nedge, sssp_max_nedge Quartiles for
 the number of input edges visited by kernel 3, see TEPS section above. sssp_mean_nedge,
 sssp_stddev_nedge Mean and standard deviation of the number of input edges
 visited by kernel 3, see TEPS section above. sssp_min_TEPS, sssp_firstquartile_TEPS,
 sssp_median_TEPS, sssp_thirdquartile_TEPS, sssp_max_TEPS Quartiles
 for the kernel 3 TEPS sssp_harmonic_mean_TEPS, sssp_harmonic_stddev_TEPS
 Mean and standard deviation of the kernel 3 TEPS.

Note: Because TEPS is a rate, the rates are compared using harmonic means.

The ****TEPS*** fields (all fields that end with “TEPS”) for Kernel 2 or Kernel

3 can be set to zero if only one kernel was run. It is permissible to run Kernel 2 and Kernel 3 on different graphs. In such situation, two outputs can be submitted, each with the ****TEPS*** for one of the kernels set to zeros.

Additional fields are permitted. Algorithm output provides a high-level sample.

```
function output (SCALE, NBFS, NSSSP, kernel_1_time, kernel_2_time, kernel_2_nedge
    , kernel_3_time, kernel_3_nedge)
printf ("SCALE: %d\n", SCALE);
printf ("NBFS: %d\n", NBFS);
printf ("construction_time: %20.17e\n", kernel_1_time);

S = statistics (kernel_2_time);
printf ("bfs_min_time: %20.17e\n", S(1));
printf ("bfs_firstquartile_time: %20.17e\n", S(2));
printf ("bfs_median_time: %20.17e\n", S(3));
printf ("bfs_thirdquartile_time: %20.17e\n", S(4));
printf ("bfs_max_time: %20.17e\n", S(5));
printf ("bfs_mean_time: %20.17e\n", S(6));
printf ("bfs_stddev_time: %20.17e\n", S(7));

S = statistics (kernel_2_nedge);
printf ("bfs_min_nedge: %20.17e\n", S(1));
printf ("bfs_firstquartile_nedge: %20.17e\n", S(2));
printf ("bfs_median_nedge: %20.17e\n", S(3));
printf ("bfs_thirdquartile_nedge: %20.17e\n", S(4));
printf ("bfs_max_nedge: %20.17e\n", S(5));
printf ("bfs_mean_nedge: %20.17e\n", S(6));
printf ("bfs_stddev_nedge: %20.17e\n", S(7));

K2TEPS = kernel_2_nedge ./ kernel_2_time;
K2N = length (K2TEPS);
S = statistics (K2TEPS);
S(6) = mean (K2TEPS, 'h');
%% Harmonic standard deviation from:
%% Nilan Norris, The Standard Errors of the Geometric and Harmonic
%% Means and Their Application to Index Numbers, 1940.
%% http://www.jstor.org/stable/2235723
k2tmp = zeros (K2N, 1);
k2tmp(K2TEPS > 0) = 1./K2TEPS(K2TEPS > 0);
k2tmp = k2tmp - 1/S(6);
S(7) = (sqrt (sum (k2tmp.^2)) / (K2N-1)) * S(6)^2;

printf ("bfs_min_TEPS: %20.17e\n", S(1));
printf ("bfs_firstquartile_TEPS: %20.17e\n", S(2));
printf ("bfs_median_TEPS: %20.17e\n", S(3));
printf ("bfs_thirdquartile_TEPS: %20.17e\n", S(4));
printf ("bfs_max_TEPS: %20.17e\n", S(5));
printf ("bfs_harmonic_mean_TEPS: %20.17e\n", S(6));
printf ("bfs_harmonic_stddev_TEPS: %20.17e\n", S(7));

S = statistics (kernel_3_time);
printf ("sssp_min_time: %20.17e\n", S(1));
printf ("sssp_firstquartile_time: %20.17e\n", S(2));
printf ("sssp_median_time: %20.17e\n", S(3));
printf ("sssp_thirdquartile_time: %20.17e\n", S(4));
printf ("sssp_max_time: %20.17e\n", S(5));
```

```

printf ("sssp_mean_time: %20.17e\n", S(6));
printf ("sssp_stddev_time: %20.17e\n", S(7));

S = statistics (kernel_3_nedge);
printf ("sssp_min_nedge: %20.17e\n", S(1));
printf ("sssp_firstquartile_nedge: %20.17e\n", S(2));
printf ("sssp_median_nedge: %20.17e\n", S(3));
printf ("sssp_thirdquartile_nedge: %20.17e\n", S(4));
printf ("sssp_max_nedge: %20.17e\n", S(5));
printf ("sssp_mean_nedge: %20.17e\n", S(6));
printf ("sssp_stddev_nedge: %20.17e\n", S(7));

K3TEPS = kernel_3_nedge ./ kernel_3_time;
K3N = length (K3TEPS);
S = statistics (K3TEPS);
S(6) = mean (K3TEPS, 'h');
%% Harmonic standard deviation from:
%% Nilan Norris, The Standard Errors of the Geometric and Harmonic
%% Means and Their Application to Index Numbers, 1940.
%% http://www.jstor.org/stable/2235723
k3tmp = zeros (K3N, 1);
k3tmp(K3TEPS > 0) = 1./K3TEPS(K3TEPS > 0);
k3tmp = k3tmp - 1/S(6);
S(7) = (sqrt (sum (k3tmp.^2)) / (K3N-1)) * S(6)^2;

printf ("sssp_min_TEPS: %20.17e\n", S(1));
printf ("sssp_firstquartile_TEPS: %20.17e\n", S(2));
printf ("sssp_median_TEPS: %20.17e\n", S(3));
printf ("sssp_thirdquartile_TEPS: %20.17e\n", S(4));
printf ("sssp_max_TEPS: %20.17e\n", S(5));
printf ("sssp_harmonic_mean_TEPS: %20.17e\n", S(6));
printf ("sssp_harmonic_stddev_TEPS: %20.17e\n", S(7));

```

References Nilan Norris, The Standard Errors of the Geometric and Harmonic Means and Their Application to Index Numbers, The Annals of Mathematical Statistics, vol. 11, num. 4, 1940. <http://www.jstor.org/stable/2235723>

Sample Driver

A high-level sample driver for the above routines is given in Algorithm driver.

```

SCALE = 10;
edgefactor = 16;
NBFS = 64;

rand ("seed", 103);

ijw = kronecker_generator (SCALE, edgefactor);

tic;
G = kernel_1 (ijw);
kernel_1_time = toc;

N = size (G, 1);
coldeg = full (spstats (G));

```

```

search_key = randperm (N);
search_key(coldeg(search_key) == 0) = [];
if length (search_key) > NBFS,
search_key = search_key(1:NBFS);
else
NBFS = length (search_key);
end
search_key = search_key - 1;

kernel_2_time = Inf * ones (NBFS, 1);
kernel_2_nedge = zeros (NBFS, 1);
kernel_3_time = Inf * ones (NBFS, 1);
kernel_3_nedge = zeros (NBFS, 1);

indeg = histc (ijw(:), 1:N); % For computing the number of edges

for k = 1:NBFS,
tic;
parent = kernel_2 (G, search_key(k));
kernel_2_time(k) = toc;
err = validate (parent, ijw, search_key(k), 0, false);
if err <= 0,
error (sprintf ("BFS %d from search key %d failed to validate: %d",
k, search_key(k), err));
end
kernel_2_nedge(k) = sum (indeg(parent >= 0))/2; % Volume/2

tic;
[parent, d] = kernel_3 (G, search_key(k));
kernel_3_time(k) = toc;
err = validate (parent, ijw, search_key (k), d, true);
if err <= 0,
error (sprintf ("SSSP %d from search key %d failed to validate: %d",
k, search_key(k), err));
end
kernel_3_nedge(k) = sum (indeg(parent >= 0))/2; % Volume/2
end

output (SCALE, NBFS, NBFS, kernel_1_time, kernel_2_time, kernel_2_nedge,
kernel_3_time, kernel_3_nedge);

```

Evaluation Criteria

In approximate order of importance, the goals of this benchmark are:

- Fair adherence to the intent of the benchmark specification
- Maximum problem size for a given machine
- Minimum execution time for a given problem size
- Less important goals:
 - Minimum code size (not including validation code)
 - Minimal development time
 - Maximal maintainability
 - Maximal extensibility

Author: Graph 500 Steering Committee

Date: 2017-06-21

2 Results

This section compares the observed Graph500 traversal performance for the MPI CPU reference implementation and the NVSHMEM GPU implementation. All runs use `SCALE=22`, `edgfactor=16`, and `NBFS=64`. Validation was disabled with `SKIP_VALIDATION=1`, so these measurements should be interpreted as performance-only runs.

The primary comparison metric is the Graph500 median traversed edges per second (median TEPS). Median kernel time is also included because it shows the direct elapsed time for a representative traversal. The CPU MPI runs used 12 MPI processes bound to hardware threads. The NVSHMEM GPU runs used one MPI process and one visible GPU.

Algorithm	Implementation	Run	Median time (s)	Median TEPS
BFS	MPI CPU reference	BFS-only	0.308311	2.17658×10^8
BFS	MPI CPU reference	BFS+SSSP binary	0.305468	2.19683×10^8
SSSP	MPI CPU reference	BFS+SSSP binary	0.759998	8.82979×10^7
BFS	NVSHMEM GPU	BFS-only	0.205414	3.26688×10^8
SSSP	NVSHMEM GPU	SSSP-only	0.0109337	1.19040×10^2

Table 1: Median Graph500 traversal performance for MPI CPU and NVSHMEM GPU runs at SCALE 22 and edgfactor 16.

For BFS, the NVSHMEM GPU implementation reaches 3.26688×10^8 median TEPS, compared with 2.17658×10^8 median TEPS for the CPU MPI BFS-only run. This is approximately a $1.50\times$ improvement in median TEPS. Compared with the BFS measurement from the combined CPU BFS+SSSP binary, the NVSHMEM GPU BFS result is approximately a $1.49\times$ improvement.

For SSSP, the CPU MPI reference run reports 8.82979×10^7 median TEPS with a median time of 0.759998 seconds. The NVSHMEM GPU SSSP-only run reports 119.04 median TEPS and a median time of 0.0109337 seconds. This result is not directly comparable to the CPU SSSP result because the same output also reports inconsistent edge-count statistics, including `thirdquartile_nedge=6.5` and `stddev_nedge=106.69224024`. The NVSHMEM SSSP result should therefore be treated as an anomalous or incomplete measurement until the SSSP edge-count and TEPS accounting are corrected.